Air-Gapped Scan

Operating Procedure

Version 1.0 · 2026-09-02

6-layer security gateway for AI-generated repositories

Contents

The acceptance gate 6
What offline coverage actually looks like 7

Purpose and the one risk that governs it

This procedure covers scanning a repository on a host with no network access.

Everything here exists to prevent a single failure: **an offline scan that reports PASS without having examined anything.** The scanners that need a network do not announce their absence loudly on their own — a missing ruleset or an unstaged database produces an empty result, and an empty result is indistinguishable from a clean one unless you check for it.

So the procedure is built around one habit: **after every scan, confirm which layers actually ran.** Step 5 is not optional paperwork. It is the step that makes the other four mean something.

Roles

Role	Works on	Responsible for
Cache operator	Connected host	Building, dating and signing the offline cache
Scan operator	Air-gapped host	Verifying the cache, running scans, reading the coverage report

The two may be the same person. The separation matters because the cache operator is the only one who can fix a missing asset, and the scan operator is the only one who can see that one is missing.

Phase 0 — Pre-flight

On the **air-gapped host**, once, before first use.

```
# 1. The pipeline itself is installed and parses
bash -n ai_transit.sh fetch_repo.sh scan_pipeline.sh && echo "scripts OK"

# 2. The test suite passes — it needs no scanning tools and no network
./tests/run_tests.sh
# Expected: ✔ 70/70 passed

# 3. Record which scanners are present
for t in betterleaks detect-secrets clamscan yara semgrep trivy \
         bandit shellcheck cppcheck hadolint checkov scancode; do
    command -v "$t" >/dev/null && echo "  present : $t" || echo "  MISSING : $t"
done

# 4. Generate the bundle integrity manifest
python3 selfcheck.py --write-manifest
```

A tool listed as `MISSING` will be reported as a WARN on every scan and will never contribute findings. Decide now whether that is acceptable; do not discover it in a report six weeks later.

Phase 1 — Build the cache (connected host)

```
./prepare_offline_cache.sh /tmp/offline-cache
```

Read its summary before continuing. It reports what it staged and what it could not.

Verify the four asset groups are present and plausible:

```
cd /tmp/offline-cache
ls -1 semgrep-rules/          # expect: owasp-top-ten, cwe-top-25,
                              #         security-audit, secrets, javascript (.yaml)
du -sh trivy-db/              # expect: several hundred MB, not a few KB
ls -1 clamav/                 # expect: *.cvd or *.cld files
ls -1 yara-rules/             # your own rules; may legitimately be empty
```

A `trivy-db` of a few kilobytes means the database did not download. That is the single most consequential thing to get right — see the note in Phase 5.

Date the cache. The scan operator cannot tell how old the data is otherwise:

```
date -u +%Y-%m-%d > /tmp/offline-cache/.cache_built_on
cat /tmp/offline-cache/.cache_built_on
```

Package and sign:

```
cd /tmp
tar -czf offline-cache.tar.gz offline-cache
sha256sum offline-cache.tar.gz | tee offline-cache.tar.gz.sha256
```

Transfer both files. Communicate the SHA-256 through a **different channel** than the archive itself — a checksum that travelled with the file it protects proves nothing about tampering in transit.

Phase 2 — Verify on arrival (air-gapped host)

Do not skip to extraction.

```
# 1. The archive matches what the cache operator sent
sha256sum -c offline-cache.tar.gz.sha256
# Expected: offline-cache.tar.gz: OK

# 2. Extract
sudo tar -xzf offline-cache.tar.gz -C /opt/ai-transit/

# 3. Every file inside is intact
cd /opt/ai-transit/offline-cache
sha256sum --check .cache_manifest.sha256 | grep -v ': OK$' || echo "all files OK"

# 4. How old is this data?
echo "cache built: $(cat .cache_built_on 2>/dev/null || echo 'UNKNOWN — reject')"
```

If step 1 or 3 fails, **stop**. Do not scan with a cache you cannot verify; request a fresh transfer.

If step 4 says `UNKNOWN`, treat the cache as untrusted for CVE purposes — you cannot judge whether its vulnerability data is current.

Phase 3 — Configure

```
# Persist for the scan operator's shell
cat >> ~/.bashrc <<'EOF'
export WORK_DIR=/opt/ai-transit
export OFFLINE=true
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
EOF
source ~/.bashrc
```

Setting `OFFLINE=true` in the environment means an operator who forgets the `--offline` flag still gets offline behaviour rather than a scan that hangs on timeouts. This is deliberate: make the safe path the default one.

Phase 4 — Run the scan

Remote URLs cannot be fetched without a network. Bring the repository to the host and scan the directory.

```
./ai_transit.sh --offline /path/to/repo
```

Common variants:

```
# CI gate — verdict only
./ai_transit.sh --offline --quiet /path/to/repo

# Audit without blocking or quarantining
./ai_transit.sh --offline --report-only /path/to/repo

# Reports only, no archive
./ai_transit.sh --offline --no-zip --no-excel /path/to/repo

# Through Docker (the wrapper forwards OFFLINE and the cache paths)
./docker-run.sh --offline /path/to/repo
```

Exit code	Meaning
`0`	PASS — approved archive in `./Good/`
`1`	FAIL — quarantined, reports in `$WORK_DIR/reports/`

Phase 5 — Confirm the scan was real

This is the step that distinguishes a trustworthy offline result from an empty one. A PASS is only meaningful if the layers that produce findings actually ran.

Every report carries a machine-readable **coverage** block stating, per layer, whether it ran. Use it — do not read the verdict alone, and do not grep warning text, because a layer can fail to run for two unrelated reasons (the tool is not installed, or its data was not staged) that produce different messages.

```
REPORT=$(ls -t "$WORK_DIR"/reports/report_*.json | head -1)

python3 - "$REPORT" <<'PY'
import json, sys
d = json.load(open(sys.argv[1]))
print("verdict          :", d["verdict"])
print("coverage_complete:", d["coverage_complete"])
print()
for layer, state in sorted(d["coverage"].items()):
    mark = "OK  " if state.startswith("ran") else "GAP "
    print(f"  {mark} {layer:26} {state}")
PY
```

Typical output for a correctly staged offline host:

```
verdict          : PASS
coverage_complete: False

  OK   L1_malware                 ran
  OK   L1_secrets_betterleaks     ran
  OK   L1_secrets_entropy         ran
  GAP  L1_ioc_yara                skipped:yara or rules directory missing
  OK   L2_owasp_cwe               ran
  OK   L3_dependency_cve          ran
  OK   L4_patterns                ran
  OK   L5_per_language_sast       ran
  OK   L6_licence                 ran
```

`L1_ioc_yara` is a legitimate gap if you have not written custom IOC rules — YARA rules are yours to supply, and having none is a configuration choice rather than a failure. Every other layer should read `ran`.

The acceptance gate

These five layers are the ones a verdict rests on. If any is not `ran`, the result is not trustworthy regardless of what it says.

```
python3 - "$REPORT" <<'PY'
import json, sys
REQUIRED = [
    "L1_secrets_betterleaks",   # credential leakage
    "L1_malware",               # malware signatures
    "L2_owasp_cwe",             # OWASP Top 10 / CWE Top 25
    "L3_dependency_cve",        # dependency vulnerabilities
    "L5_per_language_sast",     # per-language static analysis
]
cov  = json.load(open(sys.argv[1])).get("coverage", {})
gaps = [l for l in REQUIRED if not cov.get(l, "missing").startswith("ran")]
if gaps:
    print("REJECT — these layers did not run:", file=sys.stderr)
    for g in gaps:
        print(f"  {g}: {cov.get(g, 'not recorded')}", file=sys.stderr)
    sys.exit(2)
print("ACCEPT — all required layers ran")
PY
```

Exit 2 means the scan is inconclusive, not that the repository is bad. Fix the staging and scan again; do not promote the artefact on the strength of a verdict that rests on layers which never executed.

What offline coverage actually looks like

Layer	Offline status
L1 secrets — betterleaks, detect-secrets	Full
L1 malware — ClamAV	Full **if** signatures staged
L1 IOC — YARA	Full (your own rules)
L2 OWASP/CWE — Semgrep	Full **if** rulesets staged
L3 dependency CVEs	**trivy only** — see below
L4 pattern rules	Full, always
L5 per-language SAST	Full
L6 licence and copyright	Full
L6 package CVEs	Not available

The one asymmetry worth internalising: offline, the staged trivy database is your *only* dependency-CVE coverage. Connected, four tools overlap on that job (trivy, pip-audit, safety, npm audit) and one failing is survivable. Offline, there is no redundancy — if the trivy database is missing or stale, dependency vulnerabilities simply go unreported.

Phase 6 — Keep the cache current

Vulnerability data ages. A stale database reports clean results for everything published after it was built — the same outcome as not scanning, but harder to notice because the report looks normal.

Asset	Suggested cadence	Consequence of staleness
trivy database	**Weekly**	Recent CVEs unreported
ClamAV signatures	**Weekly**	Recent malware undetected
Semgrep rulesets	Monthly	Missing newer rules; existing ones still valid
YARA rules	On change	Your own IOC coverage

Repeat Phases 1–2 to refresh. Keep the previous cache until the new one is verified, so a failed transfer does not leave the host with nothing.

Check the age of the running cache at any time:

```
echo "cache built : $(cat "$OFFLINE_CACHE/.cache_built_on" 2>/dev/null || echo UNKNOWN)"
echo "today       : $(date -u +%Y-%m-%d)"
```

Phase 7 — Periodic self-check

Monthly, or after any change to the pipeline files:

```
python3 selfcheck.py --only 11.1,11.4,11.6 --format both --output selfcheck_report
```

Check 11.6 compares the bundle against its manifest. A FAIL means a pipeline file changed since the manifest was written — either an intentional update (regenerate with `--write-manifest`) or something that warrants investigation.

Checks that reach the network (11.4 Python CVE, 11.5 host OS CVE) will report SKIP or WARN offline. That is expected.

Failure reference

Symptom	Cause	Resolution
Scan hangs several minutes per layer	Running without `--offline`	Set `OFFLINE=true` or pass `--offline`
`Cannot clone ... without a network`	Remote URL in offline mode	Copy the repository locally, pass its path
Scan finishes suspiciously fast, few findings	Cache not staged	Run the Phase 5 check
`sha256sum -c` fails on arrival	Corrupt or tampered transfer	Do not extract; request a fresh transfer
`.cache_built_on` missing	Cache built without dating	Treat CVE results as untrusted; rebuild
Everything WARNs as "tool missing"	Scanners not installed on this host	Phase 0 step 3; install per INSTALL.md §6
`declare -A: invalid option`	bash < 4.0	Install bash 5

Quick reference card

```
CONNECTED HOST
  ./prepare_offline_cache.sh /tmp/offline-cache
  date -u +%Y-%m-%d > /tmp/offline-cache/.cache_built_on
  tar -czf offline-cache.tar.gz -C /tmp offline-cache
  sha256sum offline-cache.tar.gz          # send via a separate channel

AIR-GAPPED HOST
  sha256sum -c offline-cache.tar.gz.sha256          # must pass
  tar -xzf offline-cache.tar.gz -C /opt/ai-transit/
  cd /opt/ai-transit/offline-cache
  sha256sum --check .cache_manifest.sha256          # must pass

  export OFFLINE=true OFFLINE_CACHE=/opt/ai-transit/offline-cache
  ./ai_transit.sh --offline /path/to/repo

  REPORT=$(ls -t $WORK_DIR/reports/report_*.json | head -1)
  python3 gate.py "$REPORT"                         # ALWAYS do this

  exit 0 : all required layers ran -> verdict is trustworthy
  exit 2 : a required layer did not run -> verdict is inconclusive,
```

```
            fix the staging and scan again
```

Appendix — the gate as a script

Save as `gate.py` next to the pipeline:

```
#!/usr/bin/env python3
"""Reject an offline scan whose verdict rests on layers that never ran."""
import json, sys

REQUIRED = [
    "L1_secrets_betterleaks",   # credential leakage
    "L1_malware",               # malware signatures
    "L2_owasp_cwe",             # OWASP Top 10 / CWE Top 25
    "L3_dependency_cve",        # dependency vulnerabilities
    "L5_per_language_sast",     # per-language static analysis
]

report = json.load(open(sys.argv[1]))
cov    = report.get("coverage", {})
gaps   = [l for l in REQUIRED if not cov.get(l, "missing").startswith("ran")]

if gaps:
    print("REJECT — these layers did not run:", file=sys.stderr)
    for g in gaps:
        print(f"  {g}: {cov.get(g, 'not recorded')}", file=sys.stderr)
    sys.exit(2)

print(f"ACCEPT — all required layers ran (verdict: {report['verdict']})")
```

Adjust `REQUIRED` to your policy. Removing a layer from that list is a decision to accept scans that do not cover it — make it deliberately, not by omission.

Per-tool technical reference: INSTALL.md §11. Flags and environment variables: INSTALL.md §9.